

```
import { getColorNum, Num } from '../../
defaults';
import { getColor, getOpacity } from '../../
state';
import { keyToXY } from '../common';
import { createDefs, createMask, getCap }
from './defs/render';
```

```
export function updateArrows(shapes:
Element, board: Element, prefix: string):
void {
    const svg = board.querySelector('svg') ||
createNewSvg(board);
    svg.innerHTML = "";
    svg.insertAdjacentElement('afterbegin',
createDefs(prefix));
    const arrows =
shapes.querySelectorAll('g[cgHash]'); //
also includes circles which later get
filtered out
    const flip = !!board.closest('.orientation-
```

```
black');
    for (let i = 0; i < arrows.length; i++) {
        const arrowElement =
arrows[i].firstElementChild;
        if (!arrowElement ||
arrowElement.tagName.toUpperCase() !=
'LINE') continue;
        const cgHash =
arrows[i].getAttribute('cgHash');
        // cgHash format for arrows is
number,number,square,square,color
        // where square is algebraic like e4
        // and color is yellow, red, blue, or
green
        // Except for arrows which are actively
being dragged, in which case the
        // format appears to be
number,number,true,square,square,color
        // And, when multiple arrows are
pointing to the same square,
        // cgHash has an additional ,- at the
```

end

```
    const cgHashParts =  
cgHash.split(',').filter(val => val !== 'true');  
    const uci = cgHashParts[2] +  
cgHashParts[3];  
    const colorNum =  
getColorNum(cgHashParts[4]);  
    const coords = uciToCoords(uci, flip);  
    const maskId =  
createMask(uci.slice(0, 2), flip, prefix,  
board, svg);  
    const arrow = createArrow(  
    shortenTip(coords),  
    svg,  
    getCap(prefix, colorNum),  
    colorNum,  
    arrowElement.getAttribute('opacity'),  
    getArrowWidth(arrowElement),  
    );  
    if (maskId) {  
        arrow.setAttributeNS(null, 'mask',
```

```
`url(#${maskId})`);  
    }  
  }  
}
```

```
export function updateArrowColor(board:  
Element, colorNum: Num, newColor:  
string): void {  
  const arrows =  
board.getElementsByTagNameNS('http://  
www.w3.org/2000/svg', 'line');  
  for (let i = 0; i < arrows.length; i++) {  
    const arrow = arrows[i];  
    if (arrow.dataset.colorNum === `${  
{colorNum}`) {  
      arrow.setAttributeNS(null, 'stroke',  
newColor);  
    }  
  }  
  // arrow heads  
  const paths =
```

```
board.getElementsByTagNameNS('http://
www.w3.org/2000/svg', 'path');
    for (let i = 0; i < paths.length; i++) {
        const path = paths[i];
        if (path.dataset.colorNum === ` ${
colorNum}`) {
            path.setAttributeNS(null, 'fill',
newColor);
        }
    }
}
```

```
export function
updateArrowColorAll(board: Element): void
{
    const arrows =
board.getElementsByTagNameNS('http://
www.w3.org/2000/svg', 'line');
    for (let i = 0; i < arrows.length; i++) {
        const arrow = arrows[i];
        const colorNum =
```

```
Number(arrow.dataset.colorNum) as Num;
    arrow.setAttributeNS(null, 'stroke',
getColor('arrow', colorNum));
}
// arrow heads
const paths =
board.getElementsByTagNameNS('http://
www.w3.org/2000/svg', 'path');
for (let i = 0; i < paths.length; i++) {
    const path = paths[i];
    const colorNum =
Number(path.dataset.colorNum) as Num;
    path.setAttributeNS(null, 'fill',
getColor('arrow', colorNum));
}
}
```

```
export function
updateArrowOpacity(board: Element): void
{
    const svgs =
```

```
board.getElementsByTagName('svg');  
    if (svgs.length === 0) return;  
    svgs[0].style.opacity =  
getOpacity('arrow');  
}
```

```
function getArrowWidth(arrow: Element):  
number | undefined {  
    const widthStr =  
arrow.getAttribute('stroke-width');  
    if (!widthStr) return;  
    const width = Number(widthStr);  
    if (Number.isNaN(width)) return;  
    // Unadjust chessground's current arrow  
effect  
    else if (width === 0.1328125) return  
0.15625;  
    else return width;  
}
```

```
function createNewSvg(board: Element):
```

```
SVGSVGElement {  
  const svg =  
document.createElementNS('http://  
www.w3.org/2000/svg', 'svg');  
  svg.classList.add('fancyground-arrows');  
  svg.setAttribute('viewBox', '-0.5 -0.5 8 8');  
  svg.style.opacity = getOpacity('arrow');  
  
board.insertAdjacentElement('beforeend',  
svg);  
  return svg;  
}
```

```
function createArrow(  
  coords: Coords,  
  svg: Element,  
  cap: string,  
  colorNum: Num | 0,  
  originalOpacity: string,  
  width: number,  
): SVGLineElement {
```

```
const arrow =  
document.createElementNS('http://  
www.w3.org/2000/svg','line');  
const color = getColor('arrow',  
colorNum);  
arrow.setAttributeNS(null, 'x1', `_${br/>{coords.x1}}`);  
arrow.setAttributeNS(null, 'y1', `_${br/>{coords.y1}}`);  
arrow.setAttributeNS(null, 'x2', `_${br/>{coords.x2}}`);  
arrow.setAttributeNS(null, 'y2', `_${br/>{coords.y2}}`);  
arrow.setAttributeNS(null, 'marker-end',  
cap);  
arrow.setAttributeNS(null, 'stroke', color);  
if (width) arrow.setAttributeNS(null,  
'stroke-width', `${width}`);  
arrow.setAttributeNS(null, 'opacity',  
originalOpacity);  
arrow.dataset.colorNum = `${colorNum}`;
```

```
    svg.insertAdjacentElement('beforeend',  
arrow);  
    return arrow;  
}
```

```
type Coords = {  
    x1: number;  
    y1: number;  
    x2: number;  
    y2: number;  
};
```

```
function uciToCoords(uci: string, flip:  
boolean): Coords {  
    const { 'x': x1, 'y': y1 } = keyToXY(uci, flip);  
    const { 'x': x2, 'y': y2 } =  
keyToXY(uci.slice(2), flip);  
    return { x1, y1, x2, y2 };  
}
```

/// Trim the tip of a coords object so that

the arrow doesn't reach

/// all the way to the center of the square.

We do this so that multiple

/// arrows pointing to the same square
don't overlap in an ugly way.

/// In regular chessground, the trim amount
is half as much if an arrow

/// points to a square that no other arrow
points to. But here we choose

/// to use this amount of trim for all arrows
so that drawing new arrows

/// can't shorten existing arrows.

function shortenTip({ x1, y1, x2, y2 }:
Coords): Coords {

 // Treat coords as a vector from tail to tip

 const dx = x2 - x1;

 const dy = y2 - y1;

 const magnitude = Math.sqrt(dx * dx +
dy * dy);

 const scale = (magnitude - 160 / 512) /
magnitude;

```
return {  
    x1,  
    y1,  
    x2: x1 + dx * scale,  
    y2: y1 + dy * scale,  
};  
}
```

what will this coding will do?